# Zero to Hero Study Handbook: DuckDB Analytics MCP

This handbook is grounded in static analysis of this repository only. It explains how the project is built, how requests flow through the code, and how to study it effectively as a new contributor.

## Module 1: Foundations & Architecture

### What This Project Does

`duckdb-analytics-mcp` is a production-focused MCP server that exposes read-only analytics tools over local datasets using DuckDB.

Main use cases: - Let an MCP client list available datasets under a configured directory. - Describe a dataset (schema, row count, sample rows). - Execute safe read-only SQL on one selected dataset at a time through a fixed alias (`source`). - Serve results as either Markdown or JSON. - Optionally protect tool access with a static bearer token + required scope.

### Core Paradigms and Patterns Used Here

1. Layered architecture

- Transport and API layer: `src/duckdb_analytics_mcp/server.py`
- Business/service layer: `src/duckdb_analytics_mcp/service.py`
- Data discovery layer: `src/duckdb_analytics_mcp/catalog.py`
- Validation/security layer: `src/duckdb_analytics_mcp/security.py`
- Schema/contracts layer: `src/duckdb_analytics_mcp/models.py`

2. Validation-first request handling

- Tool arguments are validated with Pydantic models (`HealthRequest`, `ListDatasetsRequest`, `DescribeDatasetRequest`, `QueryDatasetRequest`).
- Invalid input becomes user-facing error payloads rather than raw tracebacks.

3. Read-only query sandboxing

- SQL is parsed with `sqlglot` AST in `SQLGuard.validate`.
- Allowed shape is a single `SELECT`/`WITH` query only.
- Dataset access is constrained to a temp view named `source`.

4. Defensive runtime controls

- DuckDB connection is configured with extension autoload/autoinstall disabled (`Settings.duckdb_config`).
- Query execution is bounded by wall-clock timeout via `_run_with_timeout`.
- Errors are sanitized in `_safe_error_message` to avoid leaking internals.

5. OOP core with functional helpers

- Primary stateful classes: `Settings`, `DatasetCatalog`, `SQLGuard`, `AnalyticsService`.
- Small pure/helper functions are used for formatting, error mapping, and serialization.

**Architecture and Component Interaction**

Key components: - CLI bootstrap (`cli.py`) - MCP server composition (`server.py`) - Analytics engine (`service.py`) - Catalog discovery cache (`catalog.py`) - SQL policy guard (`security.py`) - Response renderers (`formatter.py`)

Main request path:

```
MCP Client
   |
   | call_tool / resource read
   v
FastMCP server (build_server in server.py)
   |
   | Pydantic validation (models.py)
   v
AnalyticsService (service.py)
   |                |
   |                +--> DatasetCatalog.scan/get (catalog.py)
   |                |
   |                +--> SQLGuard.validate (security.py) for query tool
   |
   +--> DuckDB in-memory connection (_connect)
            |
            +--> register selected dataset as TEMP VIEW source
            +--> execute guarded SQL / describe / count
            +--> paginate + shape output
   |
   v
Markdown renderer (formatter.py) or JSON model dump
   |
   v
MCP response payload
```

## Module 2: Repository Map

Focus first on these files and directories.

| File/Directory Path | Primary Responsibility | Key Classes/Functions | Important Configs/Variables |
|---|---|---|---|
| `pyproject.toml` | Package metadata, dependencies, tooling config, CLI entrypoint | `[project.scripts]` `duckdb-analytics-mcp = "duckdb_analytics_mcp.cli:app"` | `requires-python`, runtime deps (`duckdb`, `mcp`, `pydantic`, `sqlglot`, `typer`) |
| `.env.example` | Runtime environment variable template | N/A | `SERVER_NAME`, `TRANSPORT`, `DATASET_DIR`, limits/timeouts, auth keys |
| `README.md` | Usage documentation and operational commands | N/A | Quickstart commands, transport modes, tool/resource names |
| `Dockerfile` | Container runtime image | `CMD ["uv", "run", "duckdb-analytics-mcp", "run", "--transport", "streamable-http"]` | `PYTHONDONTWRITEBYTECODE`, `PYTHONUNBUFFERED`, `UV_LINK_MODE` |
| `docker-compose.yml` | Local container orchestration | N/A | `HOST`, `PORT`, `TRANSPORT`, `DATASET_DIR`, data volume mount |
| `src/duckdb_analytics_mcp/cli.py` | Console/CLI entrypoint | `app`, `_callback`, `run_command`, `doctor_command` | `transport` CLI option; uses environment via `get_settings()` |
| `src/duckdb_analytics_mcp/config.py` | Typed settings and derived runtime config | `Settings`, `get_settings`, `resolved_dataset_dir`, `duckdb_config` | Limits, timeout, DuckDB config, auth URLs/scope/token |
| `src/duckdb_analytics_mcp/server.py` | FastMCP server wiring, tool/resource registration, auth, error shaping | `build_server`, `run_server`, `StaticTokenVerifier`, `_safe_error_message` | `READ_ONLY_TOOL`, `streamable_http_path="/mcp"` |
| `src/duckdb_analytics_mcp/service.py` | Core analytics operations and DuckDB execution | `AnalyticsService`, `.list_datasets`, `.describe_dataset`, `.query_dataset`, `._run_with_timeout` | `max_rows`, `max_sample_rows`, `query_timeout_seconds` |

| File/Directory Path | Primary Responsibility | Key Classes/Functions | Important Configs/Variables |
|---|---|---|---|
| `src/duckdb_analytics_mcp/dataset_catalog.py` | Dataset discovery, cache, safe path resolution | `DatasetCatalog`, `DatasetEntry`, `SUPPORTED_SUFFIXES` | `cache_ttl_seconds`, allowed suffixes: `.csv`, `.parquet`, `.json`, `.jsonl` |
| `src/duckdb_analytics_mcp/sql_ast_security.py` | SQL AST guardrails | `SQLGuard.validate`, `SQLValidationError` | `max_query_chars`, banned comments/external scans/non-query statements |
| `src/duckdb_analytics_mcp/models.py` | Request/response contracts | `ResponseFormat`, request models, result models | `extra="forbid"`; strict field constraints |
| `src/duckdb_analytics_mcp/formatters.py` | Markdown/JSON formatting for structured results | `render_health_markdown`, `render_catalog_markdown`, `render_describe_markdown`, `render_query_markdown` | ISO timestamps, code-block formatting for row/schema payloads |
| `src/duckdb_analytics_mcp/logging_utils.py` | Log configuration for safe stdio transport | `configure_logging` | Sends logs to `stderr` to avoid corrupting stdio MCP traffic |
| `data/sales_orders.csv` | Example CSV analytics dataset | N/A | Columns like `order_id`, `region`, `units`, `unit_price`, `discount_pct` |
| `data/support_tickets.jsonl` | Example JSONL dataset | N/A | JSON fields like `ticket_id`, `priority`, `status`, `resolution_hours` |
| `tests/test_security.py` | SQL guard behavior tests | `test_sql_guard_*` | Validates read-only restrictions and blocked patterns |
| `tests/test_service.py` | Service layer tests | `test_service_query_dataset_with_pagination`, timeout test | Confirms pagination fields, schema extraction, timeout behavior |
| `tests/test_server.py` | Tool-level validation/error mapping tests | `_unwrap_result_payload`, validation error tests | Confirms invalid inputs become structured error payloads |

| File/Directory Path | Primary Responsibility | Key Classes/Functions | Important Configs/Variables |
|---|---|---|---|
| `tests/test_e2e_mcp.py` | Real transport/auth end-to-end scenarios | `_call_stdio_tool`, `_call_http_tool`, auth/no-auth tests | `TOKEN`, dynamic free port setup, HTTP `/mcp` path |
| `scripts/smoke_test.py` | Manual smoke check of service layer | `main` | Emits `health` + `catalog` payload via logging |
| `scripts/execute_notebooks.py` | Notebook execution utility | `execute_notebook`, `main` | `NOTEBOOKS = ["01_mcp_server_tutorial.ipynb"]` |
| `notebooks/01_mcp_server_tutorial.ipynb` | Guided walkthrough of health/catalog/describe/query/security flows | N/A (notebook cells) | Demonstrates expected outputs and guardrail rejection |

## Module 3: Core Execution Flows

### Flow A: Server Startup (CLI to MCP Runtime)

Entry point: - CLI script binding in `pyproject.toml` points to `duckdb_analytics_mcp.cli:app`.

Step-by-step: 1. `duckdb-analytics-mcp` command loads Typer app in `src/duckdb_analytics_mcp/cli.py`. 2. `_callback()` runs first: - `settings = get_settings()` - `configure_logging(settings.log_level)` 3. `run` command calls `run_server(transport=transport)`. 4. `run_server` in `server.py`: - Loads runtime settings. - Calls `build_server(runtime)` to register resources/tools. - Chooses transport: `stdio` or `streamable-http`. 5. `FastMCP.run(...)` starts the selected transport server.

Code fragment (actual startup switch):

```
if runtime_transport == "stdio":
    server.run(transport="stdio")
    return
server.run(transport="streamable-http")
```

### Flow B: Health Tool (`duckdb_analytics_health`)

Tool handler in `build_server`: - Validates args with `HealthRequest`. - Calls `service.health()`. - Returns Markdown or JSON via `_format_response`.

Input shape:

```
{
  "response_format": "markdown | json"
}
```

Output shape (`HealthStatus`):

```
{
  "status": "ok",
  "server": "duckdb_analytics_mcp",
  "dataset_dir": "/abs/path/to/data",
  "dataset_count": 2,
  "checked_at": "ISO-8601 datetime"
}
```

### Flow C: List Datasets (`duckdb_analytics_list_datasets`)

Step-by-step: 1. Validate with `ListDatasetsRequest(limit, offset, response_format)`. 2. `AnalyticsService.list_datasets(...)`: - Calls `self._catalog.scan()` to discover supported files. - Applies bounded limit: `bounded_limit = min(limit, self._settings.max_limit)`. - Computes `has_more` and `next_offset`. 3. Returns `PaginatedDatasetsResult`. 4. Handler formats as Markdown (`render_catalog_markdown`) or JSON.

Input shape:

```
{
  "limit": 25,
  "offset": 0,
  "response_format": "markdown | json"
}
```

Output shape (`PaginatedDatasetsResult`):

```
{
  "total_count": 2,
  "count": 2,
  "limit": 25,
  "offset": 0,
  "has_more": false,
  "next_offset": null,
  "datasets": [
    {
      "name": "sales_orders.csv",
      "path": "/abs/path/data/sales_orders.csv",
      "file_format": "csv",
      "size_bytes": 1636,
      "modified_at": "ISO-8601 datetime"
    }
  ]
}
```

### Flow D: Describe Dataset (`duckdb_analytics_describe_dataset`)

Step-by-step: 1. Validate with `DescribeDatasetRequest(dataset, sample_rows, response_format)`. 2. `AnalyticsService.describe_dataset(dataset_name, sample_rows)`: - Resolves dataset via `DatasetCatalog.get`. - Caps sample rows: `min(sample_rows, self._settings.max_sample_rows)`. - Executes DB work inside `_run_with_timeout`. 3. In DB task: - `_register_source(con, entry)` creates temp view `source` from file. - `DESCRIBE SELECT * FROM source` for schema. - `SELECT COUNT(*) FROM source` for row count. - `SELECT * FROM source LIMIT ?` for sample rows. 4. Returns `DatasetDescription`.

Input shape:

```
{
  "dataset": "sales_orders.csv",
  "sample_rows": 5,
  "response_format": "markdown | json"
}
```

Output shape (`DatasetDescription`):

```
{
  "dataset": {
    "name": "sales_orders.csv",
    "path": "/abs/path/data/sales_orders.csv",
    "file_format": "csv",
    "size_bytes": 1636,
    "modified_at": "ISO-8601 datetime"
  },
  "row_count": 30,
  "columns": [
    {"name": "order_id", "data_type": "BIGINT", "nullable": "YES"}
  ],
  "sample_rows": [
    {"order_id": 1001, "region": "North", "units": 3}
  ]
}
```

### Flow E: Query Dataset (`duckdb_analytics_query_dataset`)

This is the most important runtime path.

Step-by-step: 1. Validate request with `QueryDatasetRequest`. 2. Resolve dataset with `DatasetCatalog.get(dataset_name)`. 3. Validate SQL with `SQLGuard.validate(sql)`: - Reject empty SQL. - Reject SQL over `max_query_chars`. - Reject SQL comments (`--`, `/*`, `*/` outside quoted strings). - Parse SQL (`sqlglot.parse`) and require exactly one statement. - Require parsed AST to be `exp.Query` (SELECT/WITH). - Allow only tables in `{source} U cte_names`. - Reject schema/catalog-qualified references. - Reject function

calls with names starting `read_` or ending `_scan`. 4. Execute guarded query in `_run_with_timeout`: - Register selected file as temp view `source`. - Wrap query with pagination and window-count: - `SELECT *, COUNT(*) OVER() AS __mcp_total_count_9f1b2c3d FROM (<safe_sql>) ...` - Remove internal count column from returned rows. - Compute `has_more` and `next_offset`. - If page is empty and `offset > 0`, run fallback `COUNT(*)` subquery. 5. Return `QueryResult` in requested format.

Actual internal SQL wrapper pattern:

```
SELECT *, COUNT(*) OVER() AS __mcp_total_count_9f1b2c3d
FROM (<safe_sql>) AS guarded_query
LIMIT ? OFFSET ?
```

Input shape:

```
{
  "dataset": "sales_orders.csv",
  "sql": "SELECT region, SUM(units) AS total_units FROM source GROUP BY region",
  "limit": 25,
  "offset": 0,
  "response_format": "markdown | json"
}
```

Output shape (`QueryResult`):

```
{
  "dataset": "sales_orders.csv",
  "sql": "normalized SQL without trailing semicolon",
  "total_count": 4,
  "count": 4,
  "limit": 25,
  "offset": 0,
  "has_more": false,
  "next_offset": null,
  "columns": ["region", "total_units"],
  "rows": [
    {"region": "North", "total_units": 89}
  ]
}
```

### Flow F: Resource Reads

Registered MCP resources: - `dataset://catalog` -> `dataset_catalog_resource()` - `dataset://schema/{dataset_name}` -> `dataset_schema_resource(dataset_name)`

Behavior: - Resources always return Markdown strings (via formatter). - Errors are caught and returned as `"Error: <message>"`.

### Flow G: Error and Security Handling

Error shaping in `_safe_error_message`: - `ValidationError` -> field-level validation message. - `ValueError`, `TimeoutError` -> direct safe message. - `duckdb.Error` -> generic safe DB failure message. - Other exceptions -> `"Internal server error"`.

Auth wiring: - Auth is enabled only when `static_bearer_token` is set and both URLs are provided. - `StaticTokenVerifier.verify_token` checks exact token and returns scopes list. - Required scope default is `analytics.read`.

### Flow H: Dataset Discovery and Caching

`DatasetCatalog` behavior: - Recursive scan from `dataset_dir` via `rglob("*")`. - Includes only `SUPPORTED_SUFFIXES` keys: `.csv`, `.parquet`, `.json`, `.jsonl`. - Resolves path and enforces it stays under dataset root (`relative_to` check). - Caches entries/index for `cache_ttl_seconds`. - `cache_ttl_seconds <= 0` disables caching and forces fresh scans.

## Module 4: Setup & Run Guide

### Prerequisites

- Linux or macOS
- Python `3.12.10`
- `uv`

### Clean-Machine Setup

```
git clone https://github.com/pypi-ahmad/duckdb-analytics-mcp.git
cd duckdb-analytics-mcp
uv python pin 3.12.10
uv venv --python 3.12.10 --allow-existing
uv sync --dev
cp .env.example .env
```

### Environment Variables and Config

All settings are defined by `Settings` in `src/duckdb_analytics_mcp/config.py` and can be provided via environment or `.env`.

Core runtime: - `SERVER_NAME` (default: `duckdb_analytics_mcp`) - `LOG_LEVEL` (`DEBUG|INFO|WARNING|ERROR|CRITICAL`, default `INFO`) - `HOST` (default `127.0.0.1`) - `PORT` (default `8000`) - `TRANSPORT` (`stdio|streamable-http`, default `streamable-http`)

Dataset + limits: - `DATASET_DIR` (default `data`) - `DEFAULT_LIMIT` (default `25`) - `MAX_LIMIT` (default `200`) - `MAX_QUERY_CHARS` (default `4000`) -

`QUERY_TIMEOUT_SECONDS` (default `20`) - `MAX_SAMPLE_ROWS` (default `25`) - `CATALOG_CACHE_TTL_SECONDS` (default `5`)

DuckDB runtime: - `DUCKDB_THREADS` (default `4`) - `DUCKDB_MEMORY_LIMIT` (default `1GB`)

Optional auth: - `STATIC_BEARER_TOKEN` - `AUTH_ISSUER_URL` - `AUTH_RESOURCE_SERVER_URL` - `AUTH_REQUIRED_SCOPE` (default `analytics.read`)

Auth rule: - If `STATIC_BEARER_TOKEN` is set, both `AUTH_ISSUER_URL` and `AUTH_RESOURCE_SERVER_URL` must also be set; otherwise startup raises `ValueError`.

### Typical Run Commands

HTTP transport:

```
uv run duckdb-analytics-mcp run --transport streamable-http
```

Server endpoint: - `http://127.0.0.1:8000/mcp`

Stdio transport:

```
uv run duckdb-analytics-mcp run --transport stdio
```

Deployment readiness check:

```
uv run duckdb-analytics-mcp doctor
```

### Docker Run Path

Build and run:

```
docker build -t duckdb-analytics-mcp:latest .
docker run --rm -p 8000:8000 \
  -e HOST=0.0.0.0 \
  -e PORT=8000 \
  -e TRANSPORT=streamable-http \
  duckdb-analytics-mcp:latest
```

Compose:

```
docker compose up --build
```

### Migrations/Seeding/External Services

- No database migrations are required (DuckDB runs in-memory per operation).
- No seed pipeline is required; example datasets are already present in `data/`.
- No mandatory external services are required for core runtime (auth is optional).

## Module 5: Study Plan & Practice Exercises

### Ordered Study Plan

1. Read high-level docs first:

- `README.md`
- `.env.example`
- `pyproject.toml`

2. Understand runtime configuration and boot:

- `src/duckdb_analytics_mcp/config.py`
- `src/duckdb_analytics_mcp/cli.py`
- `src/duckdb_analytics_mcp/logging_utils.py`

3. Learn server wiring and contracts:

- `src/duckdb_analytics_mcp/models.py`
- `src/duckdb_analytics_mcp/server.py`
- `src/duckdb_analytics_mcp/formatter.py`

4. Deep-dive core analytics logic:

- `src/duckdb_analytics_mcp/catalog.py`
- `src/duckdb_analytics_mcp/security.py`
- `src/duckdb_analytics_mcp/service.py`

5. Validate understanding with tests:

- `tests/test_security.py`
- `tests/test_service.py`
- `tests/test_server.py`
- `tests/test_e2e_mcp.py`

6. Use examples for concrete mental models:

- `data/sales_orders.csv`
- `data/support_tickets.jsonl`
- `notebooks/01_mcp_server_tutorial.ipynb`

### Practice Exercises

1. Trace startup end-to-end

- Task: Explain exactly what happens from `duckdb-analytics-mcp run --transport stdio` to the first tool call being available.
- Target files: `pyproject.toml`, `cli.py`, `server.py`.

2. Explain path safety in catalog scanning

- Task: Why does `DatasetCatalog` skip symlinks that escape the dataset root?
- Target files: `catalog.py`, `tests/test_catalog.py`.

3. Reconstruct SQL policy from code

- Task: List every SQL pattern that is explicitly blocked by `SQLGuard`.
- Target files: `security.py`, `tests/test_security.py`.

4. Understand pagination semantics

- Task: For query results, explain `total_count`, `count`, `has_more`, and `next_offset`, including empty-page fallback behavior.
- Target files: `service.py`.

5. Compare describe vs query execution paths

- Task: Identify which SQL statements are executed in `describe_dataset` vs `query_dataset`.
- Target files: `service.py`.

6. Map request validation to error payloads

- Task: Show how invalid request fields become safe user-facing error responses.
- Target files: `server.py`, `models.py`, `tests/test_server.py`.

7. Explain optional auth enablement

- Task: Describe the exact config condition that turns auth on and what scope is required.
- Target files: `config.py`, `server.py`, `tests/test_e2e_mcp.py`.

8. Document data contracts

- Task: Write JSON schemas (field names + types) for `PaginatedDatasetsResult` and `QueryResult`.
- Target files: `models.py`.

**Solution Outlines**

1. Startup flow outline

- CLI entrypoint resolves to `duckdb_analytics_mcp.cli:app`.
- `_callback()` loads settings and configures `loguru` to `stderr`.
- `run_command` calls `run_server`.
- `run_server` builds `FastMCP` via `build_server`, registers tools/resources, then runs selected transport.

2. Path safety outline

- `resolved_path.relative_to(self._dataset_dir)` raises if file escapes root.
- Unsafe entries are skipped in `_scan_uncached`.
- Test `test_catalog_skips_symlink_escaping_dataset_root` confirms behavior.

3. SQL policy outline

- Blocked: non-query statements (`UPDATE`, `INSERT`, etc.), multi-statements, comments, schema/catalog-qualified refs, non-`source` tables, `read_*` and `*_scan` functions, non-Identifier table expressions.

4. Pagination outline

- `count` = rows returned this page.
- `total_count` = full result size before page slicing.
- `has_more` true when `offset + count < total_count`.
- `next_offset` is next page start or `None`.
- If page empty at non-zero offset, fallback `COUNT(*)` query computes total.

5. Describe vs query outline

- `describe_dataset`: `DESCRIBE SELECT * FROM source`, `SELECT COUNT(*) FROM source`, sample `SELECT * FROM source LIMIT ?`.
- `query_dataset`: wraps validated user SQL in paginated outer query with `COUNT(*) OVER()`.

6. Validation-error mapping outline

- Tool handler calls `RequestModel.model_validate(...)`.
- `ValidationError` caught in `except Exception`.
- `_safe_error_message` converts to readable field-level message.
- `_format_error` returns `{"error": "..."}` for JSON format or `Error: ...` for Markdown.

7. Auth enablement outline

- If `static_bearer_token` is unset, auth kwargs are empty.
- If token set but issuer/resource URL missing, startup raises `ValueError`.
- With full config, `StaticTokenVerifier` accepts exact token and grants scope list containing `auth_required_scope`.

8. Contract outline

- `PaginatedDatasetsResult` fields: `total_count:int`, `count:int`, `limit:int`, `offset:int`, `has_more:bool`, `next_offset:int|None`, `datasets:list[DatasetSummary]`.
- `QueryResult` fields: `dataset:str`, `sql:str`, `total_count:int`, `count:int`, `limit:int`, `offset:int`, `has_more:bool`, `next_offset:int|None`, `columns:list[str]`, `rows:list[dict[str, object]]`.

## Learner Verification Checklist

Use this checklist before you consider yourself comfortable with the codebase:

- Can you explain the end-to-end path from CLI command to `FastMCP.run()`?
- Can you explain why logs are sent to `stderr` instead of `stdout`?

- Can you describe how `Settings.resolved_dataset_dir` is derived?
- Can you explain how dataset scanning is cached and when cache is bypassed?
- Can you list SQL constructs blocked by `SQLGuard` and why they matter?
- Can you explain how a dataset file becomes the `source` temp view in DuckDB?
- Can you explain `QueryResult.total_count` vs `QueryResult.count` using a paginated example?
- Can you describe timeout handling and where `connection.interrupt()` is attempted?
- Can you explain exactly when auth is enabled and what scope is required?
- Can you identify where markdown rendering happens for each tool output type?
- Can you trace how validation errors are transformed into safe error payloads?
- Can you describe the difference between MCP tools and MCP resources in this project?